

BTGraph Dialogue Pack Manual

BTGraph

Version 1.0

Contents

1 Overview	2
2 Installation	2
3 Core Concepts	2
3.1 Dialogue Runner	2
3.2 Dialogue UI	2
3.3 Quest and Signal Helpers	2
4 Quick Start	2
5 Sample Scene	2
6 Node Reference (Summary)	3
6.1 Core	3
6.2 Quest	3
6.3 Optional	3
7 Node Descriptions	3
7.1 Dialogue Start	3
7.2 Dialogue Line	3
7.3 Await Input	3
7.4 Choices	3
7.5 Dialogue End	4
7.6 Sequence Lines	4
7.7 Random Bark	4
7.8 Quest Offer	4
7.9 Quest Wait For Event	4
7.10 Quest Complete	4
8 Example Use Cases	4
9 EventDriven Notes	4
10 Customization Tips	5
11 Troubleshooting	5
11.1 Dialogue does not advance	5
11.2 Input System errors	5
12 API Notes	5

1 Overview

The BTGraph Dialogue Pack adds dialogue-specific nodes, a lightweight dialogue runtime, and optional quest/event helpers. It integrates with BTGraph's blackboard and execution model, and is designed to run in both Tick and EventDriven modes.

2 Installation

1. Import the Dialogue Pack into the project.
2. Ensure BTGraph Shared (runtime + editor) is present.
3. If using Input System only, make sure your demo scripts use the Input System path (already handled in samples).

3 Core Concepts

3.1 Dialogue Runner

DialogueRunner is a runtime component that executes a dialogue behavior tree and exposes events: lines, choices, end state, and signals. It wraps a **BTRunner** and manages dialogue-specific blackboard data.

3.2 Dialogue UI

SimpleDialogueUI is a lightweight UGUI presenter that renders speaker, line text, and optional choices. You can replace it with your own UI; the runtime does not require it.

3.3 Quest and Signal Helpers

The pack includes a simple quest service and a signal bus for demo and prototyping: **SimpleQuestService**, **QuestServiceProvider**, and **GameSignalBus**. These are optional and can be replaced with game-specific systems.

4 Quick Start

1. Create a BTGraph tree and add dialogue nodes (see Node Reference).
2. Add a **BTRunner** and **DialogueRunner** to the NPC.
3. Assign the tree to your controller script (example: **SampleQuestNpcController**).
4. Add **SimpleDialogueUI** and wire references.

5 Sample Scene

The demo scene shows:

- A quest offer flow using **DialogueLine**, **Await Input**, and quest nodes.
- Keyboard-driven continue and turn-in events.
- A minimal UI layout.

6 Node Reference (Summary)

6.1 Core

- Dialogue Start
- Dialogue Line
- Await Input
- Dialogue End
- Choices

6.2 Quest

- Quest Offer
- Quest Complete
- Quest Wait For Event
- Quest Signal Emit

6.3 Optional

- Sequence Lines
- Random Bark
- Memory / Checkpoint Nodes

7 Node Descriptions

7.1 Dialogue Start

Starts a dialogue session and initializes dialogue blackboard context (speaker, target, conversation id).

Use cases: entering a conversation, starting a quest flow, triggering a cutscene dialogue.

7.2 Dialogue Line

Emits a single line to the UI (speaker, text, tags, audio key).

Use cases: NPC lines, narration, UI prompts.

7.3 Await Input

Waits for a continue input (or optional timeout). This node is typically used between lines.

Use cases: tap-to-continue dialogue, timed auto-advance lines.

7.4 Choices

Presents up to four choices and executes the selected branch.

Use cases: branching dialogue, player decisions, optional quest acceptance.

7.5 Dialogue End

Ends the dialogue and returns a result (Completed/Cancelled).

Use cases: closing the dialogue UI, handing off to gameplay.

7.6 Sequence Lines

Outputs multiple lines in order from one node, optionally waiting between lines.

Use cases: compact multi-line barks, narration blocks, tutorial instructions.

7.7 Random Bark

Selects a weighted bark line from a set and emits it once.

Use cases: ambient chatter, idle comments, randomized NPC reactions.

7.8 Quest Offer

Registers a quest and emits a journal entry for the player.

Use cases: quest acceptance, objective setup.

7.9 Quest Wait For Event

Waits for a quest event to be fired (e.g., turn-in, objective completion).

Use cases: waiting on gameplay, linking dialogue to world events.

7.10 Quest Complete

Marks a quest as complete and can emit completion signals.

Use cases: wrap-up dialogue, rewards flow.

8 Example Use Cases

- **Quest Offer Flow:** Start → Line → Await Input → Quest Offer → Line → Await Input → Quest Wait → Line → End.
- **Branching Choice:** Line → Choices (Accept / Decline) → Quest Offer or End.
- **Ambient Bark:** Random Bark on interaction or proximity trigger.
- **Cutscene Dialogue:** Sequence Lines with Await Continue disabled for timed playback.

9 EventDriven Notes

Dialogue nodes are designed to be EventDriven-friendly:

- **DialogueRunner.Continue()** requests reevaluation.
- Choice selection requests reevaluation.
- Quest events request reevaluation.

10 Customization Tips

- Replace **SimpleDialogueUI** with your own UI layer.
- Replace **SimpleQuestService** with your quest backend.
- Use blackboard keys to feed dynamic speaker IDs or line text.

11 Troubleshooting

11.1 Dialogue does not advance

Ensure:

- **BTRunner** exists on the same object as **DialogueRunner**.
- The tree is assigned and valid.
- Continue input is being sent (or call `DialogueRunner.Continue()`).

11.2 Input System errors

The included samples handle both legacy input and the Input System package. If you replace them, ensure you use the correct input API.

12 API Notes

Key runtime types:

- `DialogueRunner`
- `DialogueLine`, `DialogueChoiceView`
- `DialogueContext`
- `IQuestService`